CHAPTER - 4

ANALYZING FUNCTIONAL DEPENDENCIES, REDUNDANCY AND DATA ANOMALIES

Introduction

In this chapter, the functional dependencies of each table in the **Purplle Online Beauty Shopping Management System** are identified and analyzed. Functional dependency helps determine how one attribute uniquely determines another, ensuring that the database is properly structured. This chapter also examines possible redundancy and data anomalies such as insertion, update, and deletion anomalies. Analyzing these factors helps in designing a normalized database that minimizes duplicate data, maintains consistency, and improves overall database performance.

4.1 Customer

Functional Dependency

- **Customer_ID** → First_Name, Last_Name, Gender, Email, Phone, Address, City, State, Pincode
- **Email** → Customer_ID (Unique)

Redundancy Analysis

- No redundancy because each customer has a unique Customer_ID.

Data Anomalies

- **Insertion:** Cannot store customer details without Customer_ID.
- **Update:** Updating customer information in multiple places may cause inconsistency.
- **Deletion:** Deleting a customer removes all customer information.

4.2 Category

Functional Dependency

- **Category_ID** → Category_Name, Description
- **Category_Name** → Category_ID (Unique)

Redundancy Analysis

- No duplicate category records.

Data Anomalies

- **Insertion:** Category cannot exist without Category_ID.
- **Update:** Changing category name should update only one record.
- **Deletion:** Removing a category may affect linked products.

4.3 Brand

Functional Dependency

- **Brand_ID** → Brand_Name
- **Brand_Name** → Brand_ID (Unique)

Redundancy Analysis

- Brand names are stored only once.

Data Anomalies

- **Insertion:** Brand requires Brand_ID.
- **Update:** Updating a brand affects all linked products.
- **Deletion:** Deleting a brand may orphan products.

4.4 Seller

Functional Dependency

- **Seller_ID** → Seller_Name, Email, Phone, GSTIN, Warehouse_Address, Rating
- **Email** → Seller_ID (Unique)
- **GSTIN** → Seller_ID (Unique)

Redundancy Analysis

- Seller details are stored only once.

Data Anomalies

- **Insertion:** Seller requires Seller_ID.
- **Update:** Seller information should be updated in one place.
- **Deletion:** Removing a seller affects listed products.

4.5 Product

Functional Dependency

- **Product_ID** → Product_Name, Category_ID, Brand_ID, Seller_ID, Admin_ID, Description, Expiry_Date

Redundancy Analysis

- Category, Brand, Seller, and Admin are referenced through foreign keys.

Data Anomalies

- **Insertion:** Product requires valid foreign keys.
- **Update:** Product details remain consistent.
- **Deletion:** Deleting a product affects variants and reviews.

4.6 Product_Variant

Functional Dependency

- **Variant_ID** → Product_ID, Shade_Or_Type, Price, Weight, Stock

Redundancy Analysis

- Variant-specific details are separated from Product.

Data Anomalies

- **Insertion:** Variant requires Product_ID.
- **Update:** Price or stock changes affect only one variant.
- **Deletion:** Removing a variant does not remove the product.

4.7 Cart

Functional Dependency

- **Cart_ID** → Customer_ID
- **Customer_ID** → Cart_ID (Unique)

Redundancy Analysis

- One cart per customer avoids duplicate carts.

Data Anomalies

- **Insertion:** Cart cannot exist without customer.

- **Update:** Customer-cart mapping remains consistent.
- **Deletion:** Deleting cart removes shopping session.

4.8 Cart_Item

Functional Dependency

- **CartItem_ID** → Cart_ID, Variant_ID, Quantity

Redundancy Analysis

- Prevents duplicate products within the cart.

Data Anomalies

- **Insertion:** Requires Cart_ID and Variant_ID.
- **Update:** Quantity changes occur in one record.
- **Deletion:** Removing item affects only that cart item.

4.9 Wishlist

Functional Dependency

- **Wishlist_ID** → Customer_ID
- **Customer_ID** → Wishlist_ID (Unique)

Redundancy Analysis

- One wishlist per customer.

Data Anomalies

- **Insertion:** Wishlist requires customer.
- **Update:** Customer mapping remains unique.
- **Deletion:** Wishlist items are removed.

4.10 Wishlist_Item

Functional Dependency

- **WishlistItem_ID** → Wishlist_ID, Product_ID, Date_Added

Redundancy Analysis

- Products are stored once per wishlist item.

Data Anomalies

- **Insertion:** Requires wishlist.
- **Update:** Product information remains unchanged.
- **Deletion:** Removes only selected wishlist item.

4.11 Coupon

Functional Dependency

- **Coupon_ID** → Coupon_Code, Discount_Type, Discount_Value, Valid_From, Valid_Till, Min_Order_Amount
- **Coupon_Code** → Coupon_ID (Unique)

Redundancy Analysis

- Coupon details stored once.

Data Anomalies

- **Insertion:** Coupon requires Coupon_ID.
- **Update:** Updating coupon reflects for all orders.
- **Deletion:** Past order history should remain.

4.12 Orders

Functional Dependency

- **Order_ID** → Customer_ID, Coupon_ID, Order_Date, Total_Amount, Order_Status

Redundancy Analysis

- Customer and coupon are linked using foreign keys.

Data Anomalies

- **Insertion:** Requires valid customer.
- **Update:** Order status updated in one record.
- **Deletion:** May affect payment and delivery.

4.13 Order_Details

Functional Dependency

- **Order_Detail_ID** → Order_ID, Variant_ID, Seller_ID, Quantity, Unit_Price

Redundancy Analysis

- Separates multiple products in an order.

Data Anomalies

- **Insertion:** Requires Order_ID.
- **Update:** Quantity changes affect one row.
- **Deletion:** Removes one item only.

4.14 Payment

Functional Dependency

- **Payment_ID** → Order_ID, Payment_Method, Payment_Status, Payment_Date, Amount
- **Order_ID** → Payment_ID (Unique)

Redundancy Analysis

- One payment per order.

Data Anomalies

- **Insertion:** Payment requires order.
- **Update:** Payment status updated once.
- **Deletion:** Payment history may be lost.

4.15 Delivery

Functional Dependency

- **Delivery_ID** → Order_ID, Delivery_Address, Delivery_Date, Delivery_Status
- **Order_ID** → Delivery_ID (Unique)

Redundancy Analysis

- One delivery record per order.

Data Anomalies

- **Insertion:** Delivery requires order.
- **Update:** Delivery status updated once.
- **Deletion:** Tracking history removed.

4.16 Review

Functional Dependency

- **Review_ID** → Customer_ID, Product_ID, Rating, Review_Text, Review_Date

Redundancy Analysis

- Each review stored once.

Data Anomalies

- **Insertion:** Requires customer and product.
- **Update:** Review changes affect one record.
- **Deletion:** Review history removed.

4.17 Admin

Functional Dependency

- **Admin_ID** → Admin_Name, Email, Phone, Username, Password
- **Email** → Admin_ID (Unique)
- **Username** → Admin_ID (Unique)

Redundancy Analysis

- Admin details are stored only once.

Data Anomalies

- **Insertion:** Admin requires Admin_ID.
- **Update:** Admin information updated once.
- **Deletion:** Product management history may be affected.

Conclusion

The functional dependencies for all entities in the **Purplle Online Beauty Shopping Management System** have been identified and analyzed. By using primary keys, foreign keys, and unique constraints, the database minimizes redundancy and prevents insertion, update, and deletion anomalies. This analysis ensures that the database is well-structured, consistent, and suitable for normalization, providing a strong foundation for efficient data storage and management in the e-commerce system.